

07 — SP-GiST Indexes (Space-Partitioned GiST)

"SP-GiST: hierarchical partitioning for data that naturally decomposes into non-overlapping regions."

Table of Contents

1. [Learning Objectives](#)
 2. [SP-GiST Architecture](#)
 3. [ASCII Diagram: SP-GiST Quadtree for Points](#)
 4. [SP-GiST vs GiST](#)
 5. [Supported Data Types](#)
 6. [Point Indexing with SP-GiST \(Quadtree\)](#)
 7. [Text Indexing with SP-GiST \(Radix/Trie\)](#)
 8. [Range Types with SP-GiST](#)
 9. [Inet / Cidr with SP-GiST](#)
 10. [Creating SP-GiST Indexes](#)
 11. [EXPLAIN Output for SP-GiST Scans](#)
 12. [Operator Classes Reference](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Production Scenarios](#)
 19. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain the space-partitioning approach and how it differs from GiST
 - Describe the quadtree structure used for geometric point data
 - Describe the trie/radix structure used for text data
 - Choose between SP-GiST and GiST for geometric and text data
 - Create SP-GiST indexes for points, text, ranges, and inet types
 - Identify workloads where SP-GiST outperforms GiST
-

SP-GiST Architecture

SP-GiST (Space-Partitioned GiST) is a balanced search tree framework that supports **space-partitioning** data structures. Unlike GiST (which allows overlapping bounding regions), SP-GiST partitions space into **non-overlapping** regions at each level.

Core Idea

At each node, the space is divided into a fixed number of **non-overlapping** partitions (children). Each data item belongs to exactly one partition. This creates data structures like:

- **Quadtrees**: divide 2D space into 4 quadrants recursively
- **k-d trees**: divide k-dimensional space along alternating axes

- **Radix trees / tries:** divide text by character prefixes
- **Interval trees:** divide ranges by non-overlapping sub-ranges

Key Properties

- **Non-overlapping partitions:** unlike GiST bounding boxes which can overlap
- **Fixed fan-out:** each node has exactly the same number of children (e.g., 4 for quadtree)
- **Not always balanced:** some partitions may be much denser than others
- **More efficient for "where exactly does this fit?" queries**

When SP-GiST Beats GiST

SP-GiST is typically faster than GiST when:

1. Data is **uniformly distributed** in space
2. Query shapes are **point queries or tight bounding boxes**
3. Data decomposes naturally into a hierarchy (IP prefixes, text prefixes)

GiST is typically better when:

1. Data is **clustered** in space (SP-GiST tree becomes unbalanced)
2. **Overlap queries** are common (GiST bounding boxes are more flexible)
3. The data type doesn't have a natural hierarchical decomposition

ASCII Diagram: SP-GiST Quadtree

SP-GiST Quadtree for POINT data in a 100×100 coordinate space

Data points:

A = (10, 80)
 B = (40, 90)
 C = (70, 20)
 D = (20, 30)
 E = (55, 55) ← center region

Level 0 (root): entire space [0,100] × [0,100]

Split at center (50, 50) into 4 quadrants:

```

(0,50)------(50,50)------(100,50)
|           NW           |           NE           | | |
|   A=(10,80)           |   B=(40,90)           |
|           |           |           |           |
(0,50)           (50,50)           (100,50)
|           SW           |           SE           | |
|   D=(20,30)           |   E=(55,55)           |
|           |           |   C=(70,20)           |
(0,0)------(50,0)------(100,0)
  
```

Tree structure:

```

      ROOT (space = full 100×100)
    /  |  |  \
  NW  NE  SW  SE
  |   |   |   |
  
```

```

      A=   B=   D=   ↓
(10,80)(40,90)(20,30)
          (SE node, space=[50,50]-[100,100])
          / | | \
          NW NE  SW SE
          |         |
          E=(55,55)   C=(70,20)

```

Query: WHERE point <@ BOX '(0,0),(50,50)' -- SW quadrant

Level 0: Check each quadrant

NW [0,50]×[50,100]: does not overlap SW → SKIP

NE [50,100]×[50,100]: does not overlap SW → SKIP

SW [0,50]×[0,50]: OVERLAPS → descend

SE [50,100]×[0,50]: does not overlap SW → SKIP

Level 1 (SW child): all points here are in SW quadrant

Return D=(20,30), check against exact box → matches

Result: only D

Non-overlapping partitions → EXACT pruning, no false positives!

(Unlike GiST which uses overlapping bounding boxes and has false positives)

SP-GiST vs GiST

Feature	GiST	SP-GiST
Partition style	Overlapping bounding boxes	Non-overlapping space partitions
Fan-out	Variable	Fixed (determined by opclass)
Balanced tree	Always balanced	Not always (uneven partitions)
False positives	Yes (overlap at internal nodes)	No (exact partitions)
Overlap queries	Excellent	Less efficient
Point queries	Good	Often faster (no false positives)
Clustered data	Good	Can degrade (unbalanced tree)
Uniform data	Good	Often faster
Supported types	Geometric, ranges, text, inet	Geometric points, text, ranges, inet
Custom extensions	Common	Less common

Supported Data Types

SP-GiST has built-in operator classes for:

Data Type	SP-GiST Opclass	Structure
POINT	quad_point_ops	Quadtree

BOX	box_ops (limited)	Quadtree
POLYGON	poly_ops	Quadtree
TEXT, VARCHAR	text_ops	Radix tree (trie)
INET, CIDR	inet_ops	Radix tree
int4range, int8range, etc.	range_ops	Interval tree

Point Indexing with SP-GiST (Quadtree)

```
-- Create table with 2D point data
CREATE TABLE sensors (
  id SERIAL PRIMARY KEY,
  name TEXT,
  location POINT
);

-- SP-GiST index on point
CREATE INDEX idx_sensors_location ON sensors USING SPGIST (location);

-- Queries that benefit from SP-GiST quadtree:

-- Find sensors within a box
SELECT * FROM sensors
WHERE location <@ BOX '(10,10),(50,50)';

-- Find sensors near a point (KNN with ORDER BY distance + LIMIT)
SELECT name, location <-> POINT '(30,30)' AS dist
FROM sensors
ORDER BY location <-> POINT '(30,30)'
LIMIT 10;

-- Find sensors to the left of x=50
SELECT * FROM sensors WHERE location[0] < 50; -- using array-style point access

-- Strict left of a box
SELECT * FROM sensors
WHERE location << BOX '(50,0),(100,100)'; -- strictly left of box
```

Text Indexing with SP-GiST (Trie/Radix)

SP-GiST can build a radix tree (trie) on text data. This is particularly useful for prefix-based lookups that are faster than B-tree for certain patterns.

```
-- SP-GiST index on text column
CREATE INDEX idx_users_username_spgist ON users USING SPGIST (username);
```

```
-- Queries that use SP-GiST radix tree:
-- Prefix search (same as B-tree but potentially more efficient for text)
SELECT * FROM users WHERE username LIKE 'john%';
SELECT * FROM users WHERE username >= 'john' AND username < 'joho';

-- Equality lookup
SELECT * FROM users WHERE username = 'johndoe';
```

Trie Structure for Text

Text data: 'hello', 'help', 'world', 'word', 'he'

Radix tree:

```
root
├── 'h' → subtree
│   ├── 'e' → subtree
│   │   ├── '' → leaf: 'he' (exact match)
│   │   └── 'l' → subtree
│   │       ├── 'lo' → leaf: 'hello'
│   │       └── 'p' → leaf: 'help'
│   └── ...
└── 'w' → subtree
    ├── 'o' → subtree
    │   ├── 'rld' → leaf: 'world'
    │   └── 'rd' → leaf: 'word'
```

Prefix search 'hel%':

Follow: 'h' → 'e' → 'l' → collect all descendants

Returns: 'hello', 'help' (exact prefix matching, no false positives)

Range Types with SP-GiST

SP-GiST uses an interval tree structure for range types.

```
CREATE INDEX idx_bookings_period ON bookings USING SPGIST (period);

-- Overlap query
SELECT * FROM bookings
WHERE period && TSRANGE('2024-06-01', '2024-06-30');

-- Contains point query
SELECT * FROM bookings
WHERE period @> '2024-06-15'::TIMESTAMP;
```

SP-GiST vs GiST for Range Types

- SP-GiST: typically faster for point-in-range queries (@>)
- GiST: supports more operators and handles exclusion constraints

Inet / Cidr with SP-GiST

SP-GiST builds a radix/prefix tree for IP addresses, perfect for subnet containment.

```
CREATE INDEX idx_firewall_network ON firewall_rules USING SPGIST (network);

-- Subnet containment queries
SELECT * FROM firewall_rules
WHERE network >= '192.168.1.100'::inet; -- network contains IP

SELECT * FROM firewall_rules
WHERE '192.168.1.100'::inet <= network; -- IP is contained in network
```

This is particularly useful for CIDR/network-based access control with hierarchical IP address routing.

Creating SP-GiST Indexes

```
-- Point data (quadtree)
CREATE INDEX CONCURRENTLY idx_locations_point
ON locations USING SPGIST (coords);

-- Text data (radix tree) -- explicit opclass
CREATE INDEX CONCURRENTLY idx_products_sku
ON products USING SPGIST (sku text_ops);

-- Range types
CREATE INDEX CONCURRENTLY idx_events_period
ON events USING SPGIST (period);

-- Inet/cidr
CREATE INDEX CONCURRENTLY idx_rules_cidr
ON access_rules USING SPGIST (network inet_ops);

-- With fill factor
CREATE INDEX idx_geo_data
ON geo_table USING SPGIST (location) WITH (fillfactor = 80);

-- Partial SP-GiST
CREATE INDEX idx_active_sensors
ON sensors USING SPGIST (location)
WHERE status = 'active';
```

EXPLAIN Output for SP-GiST Scans

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sensors
```

```
WHERE location <@ BOX '(0,0),(50,50)';
```

```
Bitmap Heap Scan on sensors  (cost=4.56..89.23 rows=123 width=32)
                                (actual time=0.234..1.456 rows=121 loops=1)
  Recheck Cond: (location <@ '(50,50),(0,0)')::box)
  Heap Blocks: exact=98
  Buffers: shared hit=103
  ->  Bitmap Index Scan on idx_sensors_location
        (cost=0.00..4.53 rows=123 width=0)
        (actual time=0.189..0.189 rows=121 loops=1)
        Index Cond: (location <@ '(50,50),(0,0)')::box)
Planning Time: 0.145 ms
Execution Time: 1.678 ms
```

Note: Heap Blocks: exact=98 (not "lossy") — SP-GiST is exact, no false positives for the box containment query. The Recheck is a standard precision check.

KNN Scan with SP-GiST

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT name, location <-> POINT '(50,50)' AS dist
FROM sensors ORDER BY dist LIMIT 5;
```

```
Limit  (cost=0.29..1.49 rows=5 width=40)
  ->  Index Scan using idx_sensors_location on sensors
        (cost=0.29..2345.67 rows=10000 width=40)
        Order By: (location <-> '(50,50')::point)
```

Operator Classes Reference

quad_point_ops (POINT)

```
-- Supported operators:
<@ -- point within box
<-> -- distance (for KNN with ORDER BY)
<< -- strictly left of
>> -- strictly right of
<^ -- below
>^ -- above
~= -- same point (equality)
```

text_ops (TEXT/VARCHAR)

```
-- Supported operators:
= -- equality
< -- less than (prefix tree comparison)
> -- greater than
```

```
<= -- less than or equal
>= -- greater than or equal
LIKE -- prefix matching only ('prefix%', no '%middle%')
```

range_ops (RANGE TYPES)

```
@> -- range contains element
<@ -- element contained in range
&& -- ranges overlap
<< -- strictly left of
>> -- strictly right of
```

inet_ops (INET/CIDR)

```
>>= -- network contains address
<<= -- address contained in network
&&= -- networks overlap
= -- equality
```

Common Mistakes

1. Using SP-GiST for text with arbitrary substring search

```
-- SP-GiST radix tree does NOT support:
WHERE name LIKE '%middle%' -- no substring search
WHERE name ILIKE '%john%' -- no case-insensitive substring

-- Use pg_trgm GIN for these:
CREATE INDEX ON products USING GIN (name gin_trgm_ops);
```

2. Expecting SP-GiST to work better than GiST for clustered spatial data

If all your GPS points are in New York City, the quadtree will be extremely unbalanced — the "New York City" quadrant will have millions of entries while others have none. GiST handles clustered spatial data better.

3. Using SP-GiST for range overlap queries when GiST exclusion constraints are needed

SP-GiST does not support exclusion constraints. For booking/reservation tables with no-overlap requirements, you must use GiST:

```
-- WRONG: SP-GiST doesn't support EXCLUDE
EXCLUDE USING SPGIST (resource WITH =, period WITH &&); -- ERROR

-- CORRECT:
EXCLUDE USING GIST (resource WITH =, period WITH &&);
```

4. Forgetting that SP-GiST text_ops only supports prefix LIKE


```
-- This uses SP-GiST:  
WHERE product_code LIKE 'SKU%'  
-- This does NOT use SP-GiST:  
WHERE product_code LIKE '%123%'
```

Best Practices

1. **Use SP-GiST for uniformly distributed point data** — quadtree is more efficient than GiST when points are spread evenly across the space.
2. **Use SP-GiST for IP address / CIDR prefix trees** — the radix structure perfectly matches IP address hierarchical structure.
3. **Use SP-GiST for text prefix lookups** on short codes, SKUs, prefixes — the trie structure is memory-efficient for text with common prefixes.
4. **Benchmark SP-GiST vs GiST** on your specific data before committing — the "winner" depends heavily on data distribution.
5. **Use GiST for exclusion constraints** — SP-GiST doesn't support them.
6. **Use GIN for full-text search** — SP-GiST tsvector support is minimal.

Performance Considerations

SP-GiST Index Size

SP-GiST indexes are typically similar in size to GiST for the same data. The non-overlapping partitioning structure sometimes results in slightly smaller internal pages but more levels for skewed data.

Tree Balance

SP-GiST trees can become unbalanced with clustered data. Monitor tree depth:

```
-- Approximate tree depth (no direct function, estimate from EXPLAIN)  
EXPLAIN SELECT * FROM sensors WHERE location <@ BOX '(0,0),(100,100)';  
-- Count the levels in the "Index Scan" cost
```

KNN Performance

For KNN (ORDER BY distance LIMIT N), SP-GiST quadtrees are competitive with GiST for uniformly distributed points. Both traverse the index in distance order, stopping after N results.

Interview Questions & Answers

Q1. What makes SP-GiST different from GiST?

A: SP-GiST uses non-overlapping space partitioning: at each node, the space is divided into non-overlapping regions (quadrants for 2D space, character prefixes for text). This means a data item belongs to exactly one child at each level, and search can prune subtrees with certainty. GiST uses overlapping bounding boxes,

which means a query region might overlap multiple subtree bounding boxes, requiring exploration of multiple branches and a recheck step to eliminate false positives. SP-GiST is typically faster for point lookups on uniformly distributed data; GiST is more versatile and handles clustered data and overlap queries better.

Q2. What data structures are built on top of SP-GiST?

A: SP-GiST is a generalized framework that supports several space-partitioning structures: (1) Quadtree: divides 2D space into 4 equal quadrants at each level — used for POINT data. (2) k-d tree: divides k-dimensional space along alternating axes. (3) Radix tree / trie: partitions text by character prefixes — used for TEXT and INET data. (4) Interval tree: partitions ranges into non-overlapping sub-intervals — used for range types. The specific structure is determined by the operator class used when creating the index.

Q3. Why can't SP-GiST be used for exclusion constraints while GiST can?

A: Exclusion constraints in PostgreSQL require checking whether a new row's values "conflict" with any existing row (using the exclusion operator, typically `&&` for ranges). This check must be performed at INSERT/UPDATE time and must be able to find any overlapping existing entries efficiently. The GiST access method is designed with this in mind: its consistency check (`GistConsistent`) can evaluate whether a subtree's bounding box overlaps the new value, enabling efficient conflict detection. SP-GiST's non-overlapping partition design doesn't provide the same mechanism for exclusion checks. PostgreSQL's exclusion constraint implementation is tied to the GiST access method specifically.

Q4. When would you prefer SP-GiST over a B-tree for text columns?

A: SP-GiST (using a radix/trie structure) is preferred over B-tree for text columns when: (1) The column has many strings with long common prefixes (e.g., product codes like 'SKU-2024-CATEGORY-001' through 'SKU-2024-CATEGORY-999'). The trie deduplicates shared prefixes, saving space. (2) Prefix range queries like `LIKE 'prefix%'` are frequent — the trie navigates directly to the prefix node without scanning. (3) The column has many distinct values with hierarchical structure (domain names, path strings, etc.). For simple equality lookups on short strings, B-tree is usually equally fast with less overhead.

Q5. How does SP-GiST handle highly skewed or clustered spatial data?

A: Poorly. If points cluster in one spatial region, the quadtree becomes unbalanced: the populated quadrant is subdivided many times while other quadrants remain empty. This creates a very deep path for points in the dense region (many levels to navigate) and wastes space on empty pages for sparse regions. GiST handles this better because its bounding boxes adapt to the actual distribution of data, creating a tree balanced on data density rather than spatial extent. For uniformly distributed spatial data, SP-GiST typically wins; for clustered data, GiST or PostGIS optimized structures are better.

Exercises with Solutions

Exercise 1

You have an `ip_access_logs` table with a `client_ip` INET column. Compare SP-GiST vs B-tree for the query `WHERE client_ip <= '10.0.0.0/8'` (IP is in subnet).

Solution:

```
-- B-tree cannot handle the <= (contained in network) operator
-- SP-GiST with inet_ops can:
CREATE INDEX idx_logs_ip_spgist ON ip_access_logs USING SPGIST (client_ip inet_ops);
```

```
-- GiST also works:
CREATE INDEX idx_logs_ip_gist ON ip_access_logs USING GIST (client_ip inet_ops);

-- Test query:
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM ip_access_logs
WHERE client_ip <= '10.0.0.0/8'::inet;

-- B-tree would require a Seq Scan for this operator
-- SP-GiST uses radix tree to efficiently navigate the IP prefix hierarchy
```

Production Scenarios

Scenario 1: Fleet Tracking — Points in Geographic Region

A logistics company tracks 500,000 vehicle GPS positions, updated every 30 seconds. Operations center needs to find all vehicles within a given geographic rectangle.

```
CREATE TABLE vehicle_positions (
    vehicle_id INTEGER,
    recorded_at TIMESTAMPTZ DEFAULT now(),
    location POINT
);

-- SP-GiST for spatial queries on uniformly distributed vehicle positions
CREATE INDEX CONCURRENTLY idx_vehicle_pos
ON vehicle_positions USING SPGIST (location);

-- Find vehicles in a region:
SELECT vehicle_id, location
FROM vehicle_positions
WHERE location <@ BOX '(40.7,-74.1),(40.8,-73.9)' -- Manhattan bounding box (approx
lat/lon)
    AND recorded_at > NOW() - INTERVAL '5 minutes';
-- SP-GiST prunes quadrants outside the box – no false positives

-- KNN: find 20 nearest vehicles to a dispatch point
SELECT vehicle_id, location <-> POINT '(40.75,-74.0)' AS dist
FROM vehicle_positions
WHERE recorded_at > NOW() - INTERVAL '5 minutes'
ORDER BY dist LIMIT 20;
```

Cross-References

- [05_gist_indexes.md](#) — GiST for comparison
- [04_gin_indexes.md](#) — GIN for multi-valued types
- [01_index_fundamentals.md](#) — Index types overview
- [../08_Query_Optimization/03_scan_types.md](#) — Scan types